# WorkFlow Global HRMS - Part 6: Leave Management Module

*Generated: June 23, 2026*

## 1. Introduction

Part 6 implements the Leave Management Module for the WorkFlow Global HRMS system. This module enables employees to request leave, view their leave balances, and allows managers to approve or reject leave requests. The implementation follows the modular monolith architecture with clean separation between Domain, Application, Infrastructure, and GraphQL layers.

## 2. Architecture Overview

### 2.1 Module Structure

The Leave Management Module follows the established layered architecture:

- LeaveFeature.Domain: Contains entities (LeaveRequest, LeaveBalance) and enums
- LeaveFeature.Application: Business logic, DTOs, and service interfaces
- LeaveFeature.Infrastructure: Repository implementations and DI configuration
- LeaveFeature.GraphQL: Query and mutation resolvers for API exposure

### 2.2 Domain Entities

LeaveRequest Entity:

```
public class LeaveRequest : BaseEntity
{
    public string EmployeeId { get; set; }
    public LeaveType LeaveType { get; set; }
    public DateTime StartDate { get; set; }
    public DateTime EndDate { get; set; }
    public decimal TotalDays { get; set; }
    public string? Reason { get; set; }
    public LeaveRequestStatus Status { get; set; }
    public string? ApprovalComments { get; set; }
    public string? ApprovedBy { get; set; }
    public DateTime? ApprovedOn { get; set; }
}
```

LeaveBalance Entity:

```
public class LeaveBalance : BaseEntity
{
    public string EmployeeId { get; set; }
    public LeaveType LeaveType { get; set; }
    public decimal TotalAllowed { get; set; }
    public decimal Used { get; set; }
    public decimal Pending { get; set; }
    public decimal Available { get; set; }
}
```

# WorkFlow Global HRMS - Part 6: Leave Management Module

*Generated: June 23, 2026*

## 3. Backend Implementation

### 3.1 Repository Layer

ILeaveRepository Interface:

```
public interface ILeaveRepository : IPostgresRepository<LeaveRequest>
{
    Task<IEnumerable<LeaveRequest>> GetEmployeeLeaveRequestsAsync(string employeeId);
    Task<IEnumerable<LeaveRequest>> GetPendingApprovalsAsync(string managerId);
    Task<IEnumerable<LeaveRequest>> GetLeaveRequestsByStatusAsync(
        string employeeId, LeaveRequestStatus status);
    Task<LeaveRequest?> GetLeaveRequestByIdAsync(string requestId);
}
```

ILeaveBalanceRepository Interface:

```
public interface ILeaveBalanceRepository : IPostgresRepository<LeaveBalance>
{
    Task<IEnumerable<LeaveBalance>> GetEmployeeLeaveBalancesAsync(string employeeId);
    Task<LeaveBalance?> GetLeaveBalanceAsync(string employeeId, LeaveType leaveType);
}
```

### 3.2 Application Layer - Leave Service

The LeaveService implements core business logic for leave management:

Key Methods:

- SubmitLeaveRequestAsync: Validates balance, deducts from available, adds to pending
- ApproveLeaveRequestAsync: Updates status to Approved, moves balance from pending to used
- RejectLeaveRequestAsync: Updates status to Rejected, restores balance from pending to available
- CancelLeaveRequestAsync: Allows employees to cancel pending requests

### 3.3 Leave Request Submission Flow

When an employee submits a leave request:

- 1. Calculate business days between start and end dates (excluding weekends)
- 2. Retrieve leave balance for the requested leave type
- 3. Validate sufficient available balance
- 4. Deduct requested days from available balance
- 5. Add requested days to pending balance
- 6. Create LeaveRequest with Status = Pending
- 7. Return LeaveRequestDto to caller

```
// Balance check
if (balance.Available < totalDays)
{
    throw new InvalidOperationException(
        $"Insufficient leave balance. Available: {balance.Available},
        Requested: {totalDays}");
}

// Update balance
balance.Available -= totalDays;
balance.Pending += totalDays;
await _leaveBalanceRepository.UpdateItemAsync(balance.Id, balance);
```

### 3.4 Leave Approval Flow

When a manager approves a leave request:

- 1. Validate request exists and status is Pending
- 2. Update request status to Approved
- 3. Record approver ID and comments
- 4. Move balance from pending to used
- 5. Set approval timestamp

```
// Update request
leaveRequest.Status = LeaveRequestStatus.Approved;
leaveRequest.ApprovedBy = request.ApproverId;
leaveRequest.ApprovalComments = request.Comments;
leaveRequest.ApprovedOn = DateTime.UtcNow;

// Update balance
balance.Pending -= leaveRequest.TotalDays;
balance.Used += leaveRequest.TotalDays;
```

### 3.5 Leave Rejection Flow

When a manager rejects a leave request:

- 1. Validate request exists and status is Pending
- 2. Update request status to Rejected

# WorkFlow Global HRMS - Part 6: Leave Management Module

*Generated: June 23, 2026*

-

3. Record approver ID and comments
- 4. Restore balance from pending to available
- 5. Set approval timestamp

# WorkFlow Global HRMS - Part 6: Leave Management Module

*Generated: June 23, 2026*

## 4. GraphQL API Layer

### 4.1 Queries

getMyLeaveBalances:

```
[GraphQLName("getMyLeaveBalances")]
public async Task<IEnumerable<LeaveBalanceDto>> GetMyLeaveBalancesAsync(
    [Service] ILeaveService leaveService,
    string employeeId)
{
    return await leaveService.GetMyLeaveBalancesAsync(employeeId);
}
```

getMyLeaveRequests:

```
[GraphQLName("getMyLeaveRequests")]
public async Task<IEnumerable<LeaveRequestDto>> GetMyLeaveRequestsAsync(
    [Service] ILeaveService leaveService,
    string employeeId)
{
    return await leaveService.GetMyLeaveRequestsAsync(employeeId);
}
```

getPendingLeaveApprovals:

```
[GraphQLName("getPendingLeaveApprovals")]
public async Task<IEnumerable<LeaveRequestDto>> GetPendingLeaveApprovalsAsync(
    [Service] ILeaveService leaveService,
    string managerId)
{
    return await leaveService.GetPendingApprovalsAsync(managerId);
}
```

### 4.2 Mutations

submitLeaveRequest:

```
[GraphQLName("submitLeaveRequest")]
public async Task<LeaveRequestDto?> SubmitLeaveRequestAsync(
    [Service] ILeaveService leaveService,
    SubmitLeaveRequest request)
{
    return await leaveService.SubmitLeaveRequestAsync(request);
}
```

approveLeaveRequest:

```
[GraphQLName("approveLeaveRequest")]
public async Task<LeaveRequestDto?> ApproveLeaveRequestAsync(
    [Service] ILeaveService leaveService,
    ApproveLeaveRequest request)
{
    return await leaveService.ApproveLeaveRequestAsync(request);
```

```
}
```

rejectLeaveRequest:

```
[GraphQLName("rejectLeaveRequest")]
public async Task<LeaveRequestDto?> RejectLeaveRequestAsync(
    [Service] ILeaveService leaveService,
    RejectLeaveRequest request)
{
    return await leaveService.RejectLeaveRequestAsync(request);
}
```

## 5. Frontend Implementation

### 5.1 Leave Dashboard

The Leave Dashboard (/leave) provides:

- Leave Balance Cards: Display Total, Used, Pending, and Available days for each leave type
- Color-coded cards: Teal (Casual), Orange (Sick), Blue (Personal)
- Icons: Umbrella (Casual), Heart (Sick), User (Personal)
- Request Leave button: Opens modal for submitting new requests

### 5.2 Leave Request Form

The Request Leave modal includes:

- Leave Type dropdown: Casual, Sick, Personal
- Start Date picker: Minimum date is today
- End Date picker: Minimum date is start date
- Reason textarea: Optional field for leave justification
- Validation: Ensures end date is after start date
- Balance check: Backend validates sufficient available balance

### 5.3 My Leave Requests View

Displays all leave requests submitted by the employee:

- Leave type and status badge (Pending/Approved/Rejected)
- Date range and total days
- Reason for leave
- Approval comments (if any)
- Cancel button for pending requests

### 5.4 Manager Approval Dashboard

Available for Manager and HR roles:

- Separate tab showing pending approvals from team members
- Employee name, leave type, and dates
- Reason for leave request
- Review button opens approval modal
- Approve/Reject actions with optional comments

### 5.5 GraphQL Integration

Frontend GraphQL queries:

```
export const GET_MY_LEAVE_BALANCES = gql`
  query GetMyLeaveBalances($employeeId: String!) {
    getMyLeaveBalances(employeeId: $employeeId) {
      id
      leaveType
```

```
      totalAllowed
      used
      pending
      available
    }
  }
`;
```

Frontend GraphQL mutations:

```
export const SUBMIT_LEAVE_REQUEST = gql`
  mutation SubmitLeaveRequest($request: SubmitLeaveRequestInput!) {
    submitLeaveRequest(request: $request) {
      id
      leaveType
      startDate
      endDate
      totalDays
      status
    }
  }
`;
```

WorkFlow Global HRMS - Part 6: Leave Management Module

*Generated: June 23, 2026*

# 6. Database Configuration

## 6.1 Entity Configuration

LeaveRequest and LeaveBalance entities are configured in LeaveEntityConfiguration:

- Table names: LeaveRequests, LeaveBalances
- Primary keys: Id (string)
- Foreign keys: EmployeeId references Employees table
- Indexes: Created on EmployeeId and Status for query optimization
- Enums: LeaveType and LeaveRequestStatus stored as strings

## 6.2 Seed Data

Initial leave balances are seeded for demo employees:

- Casual Leave: 12 days per year
- Sick Leave: 10 days per year
- Personal Leave: 5 days per year
- All balances start with Used=0, Pending=0, Available=TotalAllowed

# WorkFlow Global HRMS - Part 6: Leave Management Module

*Generated: June 23, 2026*

## 7. Business Logic & Validation

### 7.1 Business Days Calculation

The system calculates business days excluding weekends:

```
private decimal CalculateBusinessDays(DateTime startDate, DateTime endDate)
{
    decimal days = 0;
    var currentDate = startDate.Date;
    var end = endDate.Date;

    while (currentDate <= end)
    {
        // Exclude weekends (Saturday and Sunday)
        if (currentDate.DayOfWeek != DayOfWeek.Saturday &&
            currentDate.DayOfWeek != DayOfWeek.Sunday)
        {
            days++;
        }
        currentDate = currentDate.AddDays(1);
    }
    return days;
}
```

### 7.2 Balance Validation

Before submitting a leave request:

- System retrieves current leave balance for the requested type
- Validates that Available balance >= Requested days
- Throws InvalidOperationException if insufficient balance
- Prevents negative balances through validation

### 7.3 Status Transitions

Valid status transitions:

- Pending -> Approved: Manager approves request
- Pending -> Rejected: Manager rejects request
- Pending -> Rejected: Employee cancels request
- Invalid: Cannot approve/reject non-pending requests

### 7.4 Balance State Management

Leave balance states:

- Available: Days employee can request
- Pending: Days in submitted but not approved requests
- Used: Days in approved requests
- Formula: TotalAllowed = Available + Pending + Used

# WorkFlow Global HRMS - Part 6: Leave Management Module

*Generated: June 23, 2026*

## 8. Technical Interview Questions

### 8.1 Architecture Questions

Q1: Why separate LeaveRequest and LeaveBalance entities?

A: Separation of concerns. LeaveRequest tracks individual requests with approval workflow, while LeaveBalance maintains aggregate state. This allows historical tracking of requests while efficiently querying current balance state.

Q2: How does the system prevent race conditions in balance updates?

A: The repository pattern with Entity Framework provides transaction support. Each balance update is atomic. For high-concurrency scenarios, we could add optimistic concurrency with row versioning or pessimistic locking.

### 8.2 Business Logic Questions

Q3: How would you handle overlapping leave requests?

A: Add validation in SubmitLeaveRequestAsync to check for existing approved/pending requests with overlapping dates. Query LeaveRequests where EmployeeId matches and dates overlap, reject if found.

Q4: How would you implement leave carry-forward?

A: Add a scheduled job that runs annually to: 1) Calculate unused leave (Available balance), 2) Apply carry-forward rules (e.g., max 5 days), 3) Reset Used to 0, 4) Update TotalAllowed with new allocation + carry-forward.

### 8.3 Performance Questions

Q5: How would you optimize the pending approvals query for large teams?

A: 1) Add composite index on (ManagerId, Status, StartDate), 2) Implement pagination, 3) Cache manager-employee relationships, 4) Consider materialized view for frequently accessed approval queues.

Q6: How would you handle bulk leave approvals?

A: Create a new mutation acceptBulkLeaveRequests that: 1) Accepts array of request IDs, 2) Validates all requests in single query, 3) Updates all in single transaction, 4) Batch updates balances, 5) Returns summary of successes/failures.

### 8.4 GraphQL Questions

Q7: Why use DTOs instead of returning domain entities directly?

A: DTOs provide: 1) API stability (domain changes don't break API), 2) Security (hide sensitive fields), 3) Performance (include only needed fields), 4) Flexibility (combine data from multiple sources).

Q8: How would you implement real-time notifications for leave approvals?

A: Use GraphQL subscriptions: 1) Create LeaveRequestUpdated subscription, 2) Publish events when status changes, 3) Filter by employeeId, 4) Frontend subscribes and shows toast notifications.

## 9. Testing Strategy

### 9.1 Unit Tests

LeaveService unit tests should cover:

- SubmitLeaveRequest: Valid request, insufficient balance, invalid dates
- ApproveLeaveRequest: Valid approval, invalid status, non-existent request
- RejectLeaveRequest: Valid rejection, balance restoration
- CalculateBusinessDays: Various date ranges, weekend handling

### 9.2 Integration Tests

Repository integration tests:

- GetPendingApprovalsAsync: Returns only pending requests for manager's team
- GetEmployeeLeaveRequestsAsync: Returns all requests ordered by date
- Balance updates: Verify atomic updates across request lifecycle

### 9.3 End-to-End Tests

Complete workflow tests:

- Employee submits leave -> Manager approves -> Balance updated correctly
- Employee submits leave -> Manager rejects -> Balance restored
- Employee cancels pending request -> Balance restored
- Insufficient balance -> Request rejected with error message

## 10. Security Considerations

### 10.1 Authorization

Access control rules:

- Employees can only view/submit their own leave requests
- Managers can only approve requests from their direct reports
- HR can approve any request
- Employees cannot approve their own requests

### 10.2 Data Validation

Input validation:

- Date validation: End date must be after start date
- Date validation: Start date cannot be in the past
- Balance validation: Cannot request more than available
- Status validation: Only pending requests can be approved/rejected

### 10.3 Audit Trail

Tracking changes:

- CreatedAt: Timestamp when request was submitted
- ApprovedBy: ID of manager who approved/rejected
- ApprovedOn: Timestamp of approval/rejection
- ApprovalComments: Reason for approval/rejection

## 11. Future Enhancements

### 11.1 Advanced Features

- Leave calendar view: Visual representation of team leave schedules
- Leave policies: Configurable rules per department/country
- Holiday calendar: Exclude public holidays from business days calculation
- Leave encashment: Convert unused leave to salary
- Comp-off management: Track overtime and compensatory leave

### 11.2 Notifications

- Email notifications: Send emails on request submission/approval/rejection
- Push notifications: Real-time updates via web push
- Reminder notifications: Remind managers of pending approvals
- Balance alerts: Notify when balance is low

### 11.3 Reporting

- Leave utilization reports: Track usage patterns by department
- Approval time metrics: Measure manager response times
- Leave forecasting: Predict future leave patterns
- Compliance reports: Ensure policy adherence

### 11.4 Integration

- Calendar integration: Sync approved leave to Outlook/Google Calendar
- Payroll integration: Deduct unpaid leave from salary
- Attendance integration: Mark leave days in attendance records
- Project management: Alert project managers of team member leave

## 12. Conclusion

Part 6 successfully implements a comprehensive Leave Management Module for the WorkFlow Global HRMS system. The implementation follows clean architecture principles with clear separation of concerns across Domain, Application, Infrastructure, and GraphQL layers.

Key achievements include:

- Robust balance management with atomic updates
- Complete approval workflow with status tracking
- Business days calculation excluding weekends
- Comprehensive validation and error handling
- Responsive mobile-first UI with role-based views
- GraphQL API with queries and mutations
- Manager approval dashboard for team oversight

The module is production-ready and can be extended with additional features such as holiday calendars, leave policies, notifications, and advanced reporting capabilities.